University of British Columbia
Electrical and Computer Engineering
Introduction to Microcomputers EECE259

Lecture 17: I/O Ports in the 8051 Microcontroller

Dr. Jesús Calviño-Fraga
Department of Electrical and Computer Engineering, UBC
Office: KAIS 3024
E-mail: jesusc@ece.ubc.ca
Phone: (604)-827-5387

June 11, 2014

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Processor of the Apollo Guidance Computer at the National *Air and Space* Museum in Washington DC. (That is my hand!)

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Objectives

- Perform digital I/O using the 8051.
- Add external ports to the 8051.
- Expand the internal ports of the 8051.
- Use 7-segment displays to show numbers.
- Read push buttons.
- Perform keyboard scanning for matrix keypads.
- Understand and solve the problem of contact bounce.
- Connect multiple 7-segment displays in multiplexed mode to the 8051.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Input/Output Modes

- A microcontroller that does not interact with the external world is basically useless!
- The microcontroller may need to interact with external peripheral devices such as: switches, LEDs, screens, LCD displays, speakers, ADC & DAC, modems, disk drives, etc.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Input/Output Modes

- Peripheral devices are connected to the microcontroller using interface circuitry:
  - A digital circuit whose function is to coordinate data transfers between the CPU and I/O devices.
  - The interface circuit may consist of control registers, status, and control registers, as well as latches for data output and 3-state buffers for data input.
  - The interface circuit has pins that are connected to the CPU and I/O port pins that are connected to the peripheral devices.
  - Interface circuits usually have a chip enable signal input, when asserted, it allows the interface circuit to communicate with the CPU.

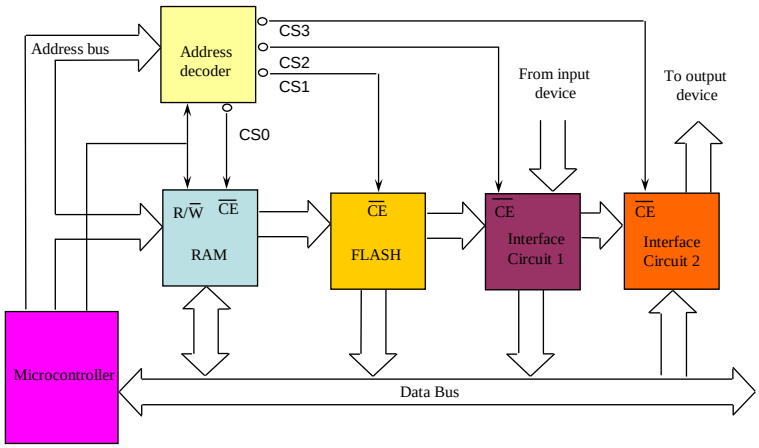
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Input/Output Alternatives

- Unconditional I/O transfer: Timing or synchronization is not critical.
- Polled I/O transfer: we need to wait for data or devices to be ready before reading or writing to them. The device may provide a signal, or we may have to wait some specific time.
- Handshake I/O transfer: The device informs the microcontroller when data is ready. Very efficient! Uses interrupts. More on this when we cover interrupts
- DMA I/O transfer: Very, very efficient!. The I/O values are put/taken from memory directly without CPU intervention. (Very hard to implement with microcontrollers, not covered in this course ☹)

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports



A microcontroller system with RAM, ROM, and I/O Ports

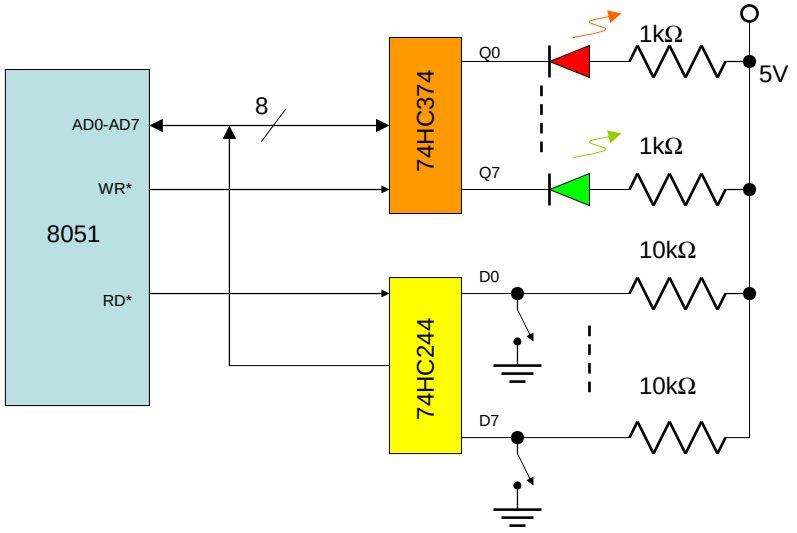
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports

- Example: writing data to LEDs and reading data from switches
  - Use an 8 bit latch as output circuit: 74HC373 or 74HC374
  - Use an 8 bit tri-state buffer as input device: 74HC244 or 74HC245
- The following approach is the simplest possible with the 8051: doesn't require address / data de-multiplexing (we save a latch).

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports

- Both the enable for the 74HC374 and 74HC244 are active low.
- Writing to <u>any</u> external address will write to the 74HC374.
- A zero turns a LED on. If you want to turn the LEDs on with one, use an inverter as they do in the course textbook.
- Reading from <u>any</u> external address will get data from the 74HC244
- When a switch is closed we read a zero.
- We use the ***movx*** instruction to access the ports as it is the only instruction that asserts RD* and WR* automatically…

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

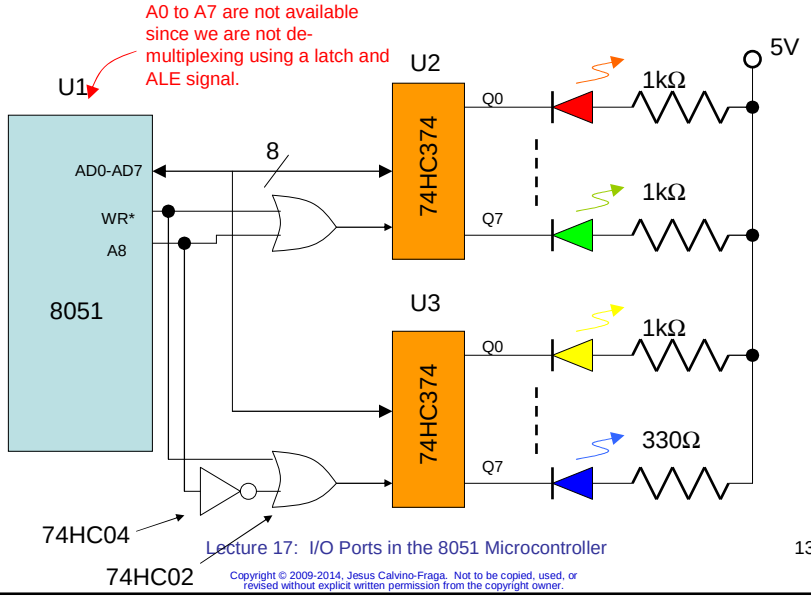
External Memory Mapped I/O ports

```
mov dptr, #0 ; Any address will do!
movx a,@dptr ; Read inputs of 74HC244 into accumulator
mov 55H, a ; Save what we read somewhere!
clr a
movx @dptr, a ; Turn on all LEDs!
```

- What if we need, say, more LEDs?
  - We have to decode the external memory map to select different devices:

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports

- U2 will be enabled when A8=0 and WR*=0.
- U3 will be enabled when A8=1 and WR*=0.
- Since we are not using any other address pin(s) in the decoding logic, it doesn't matter what value we use for those unused bits in the DPTR register.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

External Memory Mapped I/O ports

```
mov dptr, #0000H ; We make A8 explicitly zero
mov a, #55H
movx @dptr, a ; on-off-on-off-on-off-on-off of U2 LEDs

mov dptr, #0100H ; We make A8 explicitly one
mov a, #0FH
movx @dptr, a ; on-on-on-on-off-off-off-off of U3 LEDs
```

- You can buy decoder chips with a number of active low outputs. Two examples are the 74HC138 (3 to 8 decoder) and 74HC139 (dual 2 to 4 decoder).
- The 138 IC is used in figure 6.19 of the course textbook.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Internal 8051 I/O ports

- The standard 8051 has **FOUR** 8-bit bidirectional I/O ports.
- To use a pin on the original 8051 as an input you **MUST** first write a ‘1’ to it.
- All pins of port 0 of the standard 8051 need a pull-up resistor (around 10k) before used as outputs.
- All standard I/O pins are bit and byte addressable. For example:

```
setb P1.0
clr P2.7
mov P0, #0C0H
```

- All I/O pins may serve multiple functions. For example P3.4 can also be used as the input for Timer 0.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Configuring the Ports for Input/Output

- Newer 8051 variants have ports that can handle more current, but they need to be configured first.
- For example, to configure the ports in the DE2-8052 soft-core we use SFRs P0MOD, P1MOD, P2MOD, and P3MOD:

```
mov P0MOD, #00000010b ; Make P0.1 output
mov P1MOD, #0ffH ; All pins of P1 are now outputs
mov P2MOD, #00001111b ; P2.0 to P2.3 outputs, P2.4 to P2.7
inputs
```

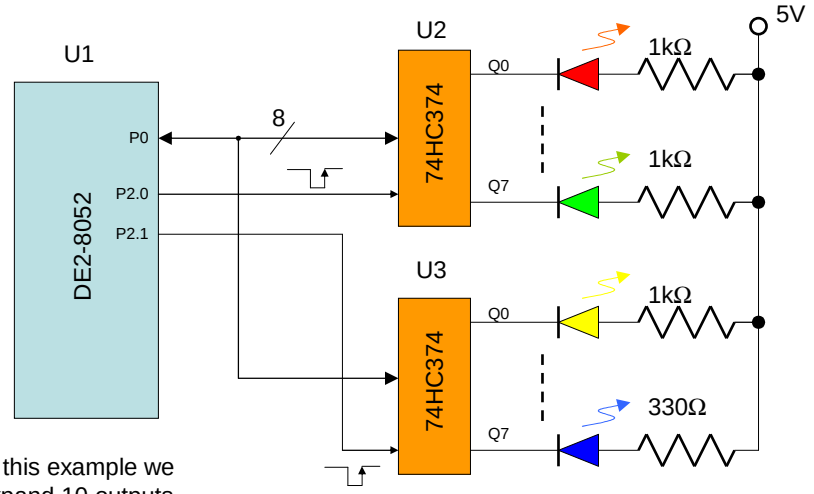

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Expanding the Internal Ports

- Many of new 8051 derivatives (small and cheap) do not have an external address/data bus.
- If you need more digital I/O there are basically two options:
  - Use SPI or I^2C peripherals. (Future lecture!). Works great and nowadays you can get wackiest SPI or I^2C peripherals at very low cost!
  - Use digital logic to expand the available ports. Very similar to memory/IO expansion with address data bus as explained above.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Expanding the Internal Ports



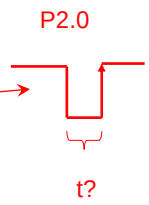
In this example we expand 10 outputs into 16 outputs!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Expanding the Internal Ports

```
; Configure P0 and P2
mov P0MOD, #0FFH
mov P2MOD, #03H

; Write R3 to U2
mov P0, R3
; Strobe the value into U2
clr P2.0
setb P2.0
; Write 55H to U3
mov P0, #55H
; Strobe the value into U3
clr P2.1
setb P2.1
```



f=11.11MHz, 1 clock per cycle, two cycle instruction: 180ns

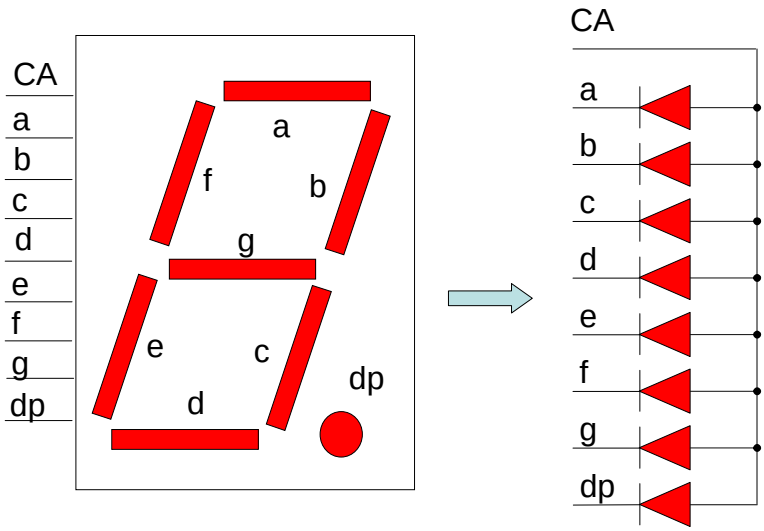
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: Writing to Seven Segment LED displays

- Add two 7-segment LED displays to ports 0 and 2 of the DE2-8052 microcontroller. The clock of the microcontroller has a frequency of 11.11MHz and it takes one clock period per machine cycle. Write a program to increment a counter approximately every second from 00 to 59 (in BCD). When the count reaches 60 reset it to zero and keep going.

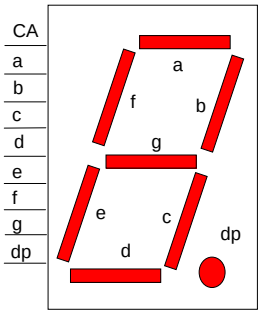
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

7-Segment Display



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

7-segment display look-up table

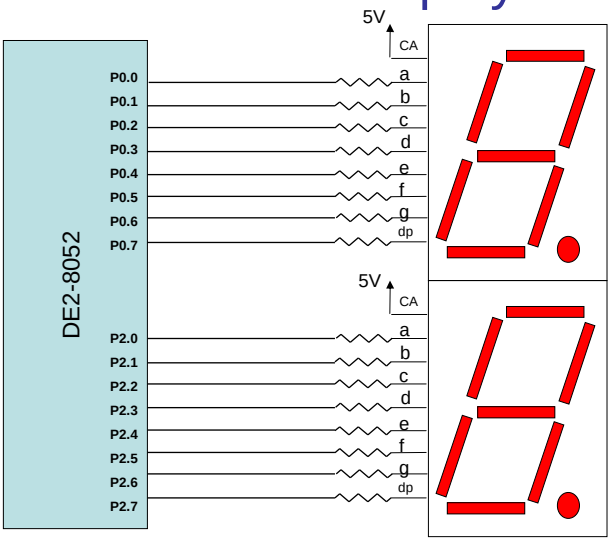


#	dp	g	f	e	d	c	b	a	HEX
0									
1									
2									
3									
4									
5									
6									
7									
8									
9									

Zero is on!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: Writing to 7-Segment LED displays



MSD

All resistors are 1kΩ.

LSD

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: code

```
$MODDE2
org 0
   ljmp mycode

; Look-up table...
mytable:
    DB 0C0H, 0F9H, 0A4H, 0B0H, 099H
    DB 092H, 082H, 0F8H, 080H, 090H
Wait1Sec:
;11.11MHz, 1 clock per cycle: 0.090us
    mov R2, #60
L3: mov R1, #250
L2: mov R0, #250
L1: djnz R0, L1 ; 3 machine cycles-> 3*90ns*250=67.5us
    djnz R1, L2 ; 67.5us*250=0.0169s
    djnz R2, L3 ; 0.0169s*60=1s (approximately)
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: code

```
mycode:
    mov SP, #7FH
    mov P0MOD, #0FFH
    mov P2MOD, #0FFH
    mov B, #0 ; counter
    mov dptr, #mytable
forever:
; Display MSB
    mov A, B
    swap A
    anl A, #0FH
    movc A, @A+dptr
    mov P0, A
; Display LSB
    mov A, B
    anl A, #0FH
    movc A, @A+dptr
    mov P2, A
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: code

```
; Do the one second delay
    lcall Wait1Sec
; Increment counter (register B) and convert to BCD
    mov A, B
    add A, #1
    da A
    mov B, A
; Check if count is 60
    cjne A, #060H, forever
    mov B, #0
    sjmp forever
END
```

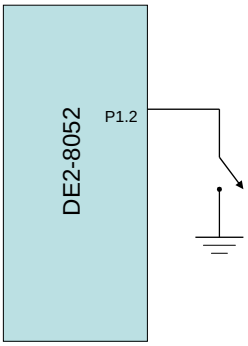

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2: Reading Push Buttons

- Before using a pin for input we need to configure it:
  - Original 8051: Write ‘1’ to the pin to be used as input.
  - Newer 8051s: configure the pin as input using designated SFRs .
- In the original 8051 any pin can be used as output or input. In newer 8051s some pins can be only input and/or outputs.
- In the original 8051 pins in the same port can be independently used as inputs or outputs. For example pin P0.0 can be used as input, while P0.1 can be used as output!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2: Reading Push Buttons




```
mov A, P1MOD
anl a, #11111011b
mov P1MOD, a
.
.
.
.
jnb P1.2, ButtonPressed
jb P1.2, ButtonNotPressed
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Matrix Keypads

- Inexpensive and commercially available.
- Can be get to work with only 2*sqrt(keys) I/O pins from the microcontroller. For example, to scan a 16-key keypad we would need 2*sqrt(16)=2*4=8 I/O pins.
- If using external logic, even fewer pins may be required!

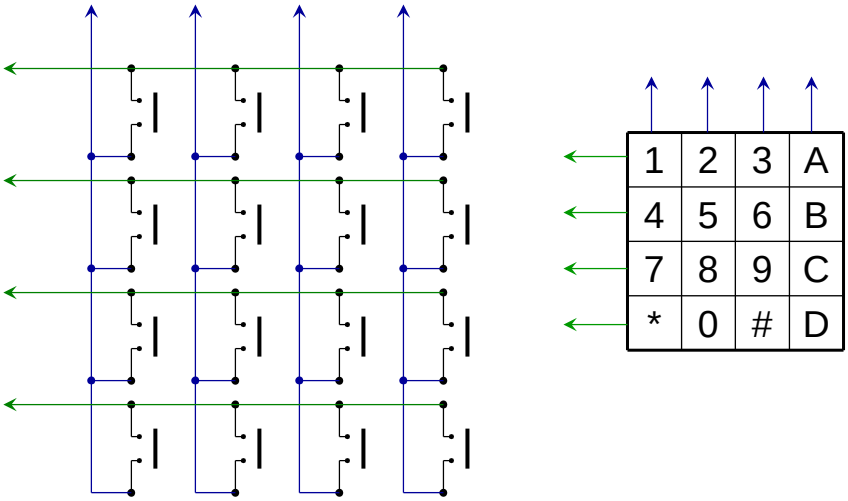
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Matrix Keypads

DigiKey par number GH5011-ND. About CA$ 15.48 each. This is the Series 96 keypads from Grayhill.

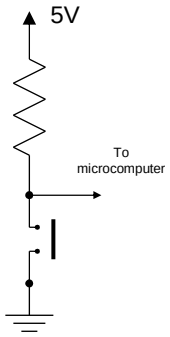
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

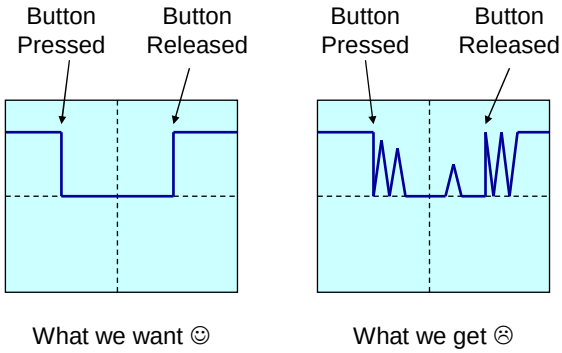
Matrix Keypads



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Problem: Contact Bounce





The time the contact bounces
can be as long as 50ms!

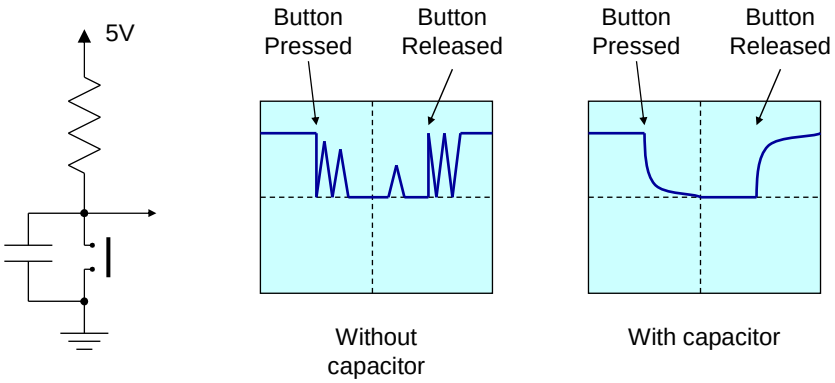
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Hardware Debouncing

- If you really have to (software debouncing is way better) these are some hardware debouncing techniques:
  - Integrating RC: Cheap and mostly safe.
  - SR latch: needs expensive switches.
  - Retrigerable monostable: Glitch sensitive.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

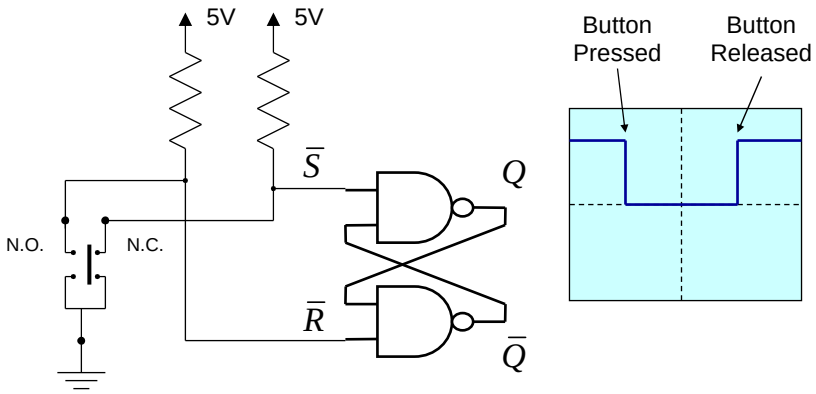
Integrating RC Debouncing



Potential problem: What is the state of the output at 'power on'?

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

SR Latch Debouncing

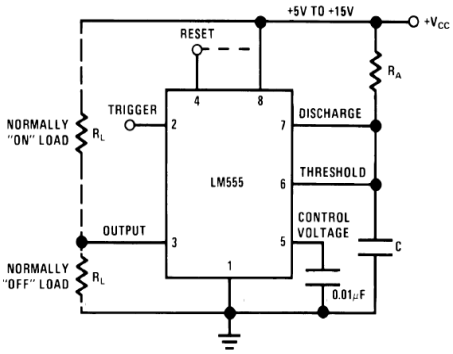


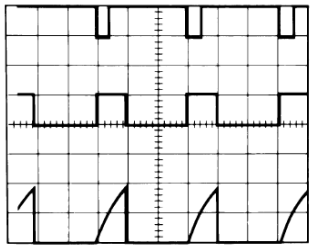
Very good, but expensive.

N.O.: Normally open
N.C.: Normally closed

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Retrigerable Monostable Debouncing





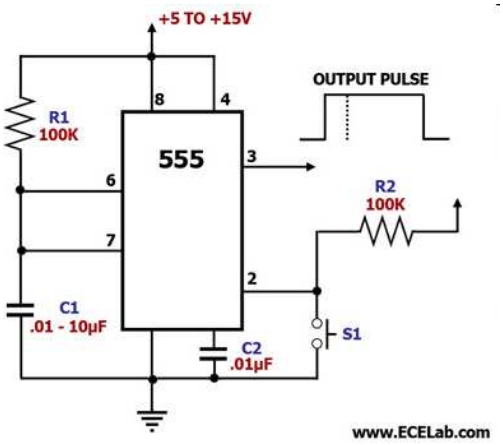
V_{CC} = 5V
TIME = 0.1 ms/DIV.
R_A = 9.1kΩ
C = 0.01μF

Top Trace: Input 5V/Div.
Middle Trace: Output 5V/Div.
Bottom Trace: Capacitor Voltage 2V/Div.

Taken from National Semiconductor's LM555 datasheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Retrigerable Monostable Debouncing



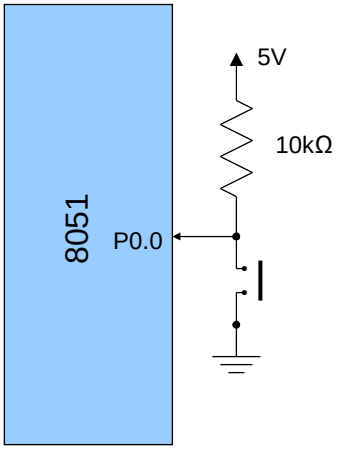
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Reasons to Avoid Hardware Debouncing

- It costs more than software debouncing!
- Hitting or shaking the pushbutton (or the case it is attached to) may generate a false trip.
- It takes more circuit space. Harder to wire.
- Software debouncing is much, much better!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Software Debouncing

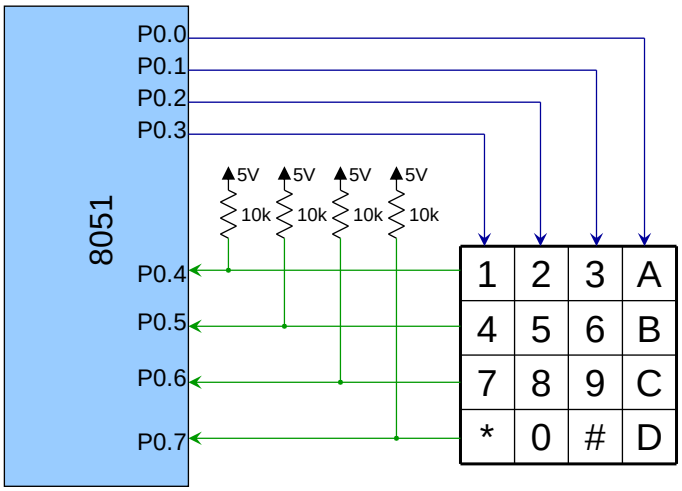



```
setb P0.0 ; Before using as input...
jb P0.0, not_pressed
lcall Wait50ms ; Wait and check again
jb P0.0, not_pressed
; Wait for the button to be released
L0: jnb P0.0, L0
sjmp pressed
```

This technique is called *wait-and-see* in many textbooks

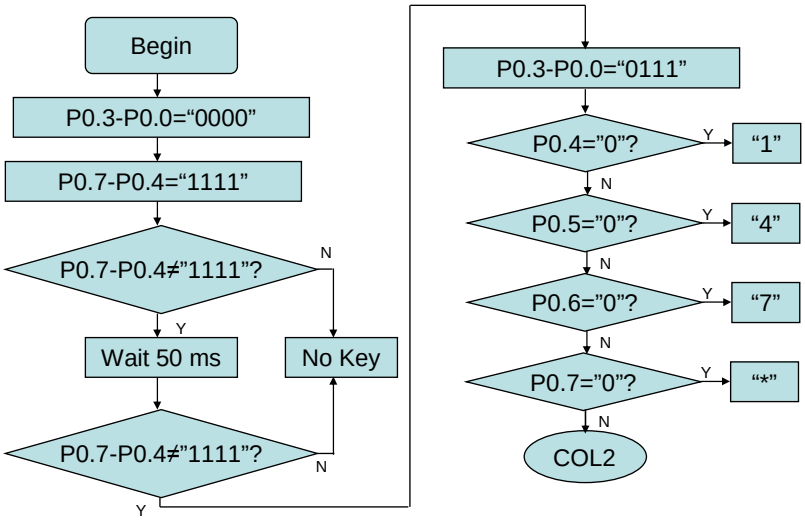
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad



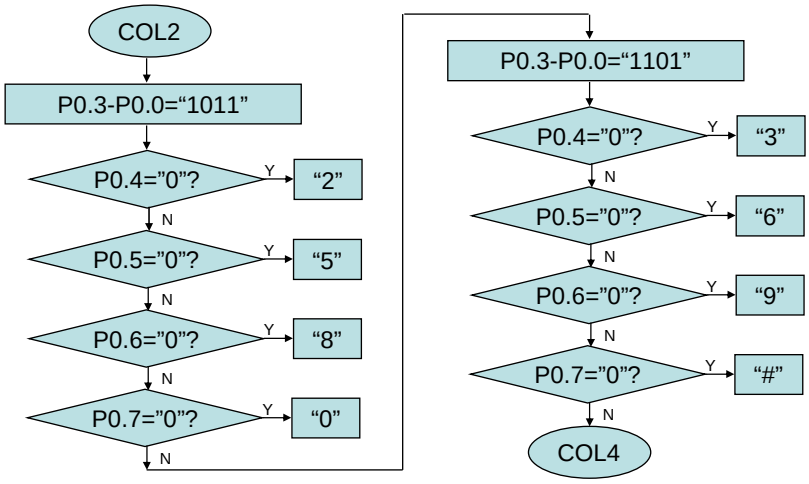
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad



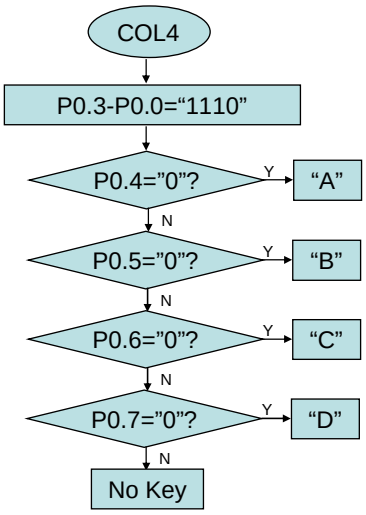
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad

```
Scan_Keypad:
   mov P0MOD, #00001111B
   mov P0, #11110000B
   mov A, P0
   anl A, #11110000B
   cjne A, #11110000B, L1
L0:
   mov A, #0 ; No key detected
   ret
L1:
   lcall Wait50ms
   mov A, P0
   anl A, #11110000B
   cjne A, #11110000B, L2
   sjmp L0
```

```
L2:
   mov P0, #11110111B
   jb P0.4, L3
   mov a, #'1' ; ASCII code
   ret
L3:
   jb P0.5, L4
   mov a, #'4'
   ret
L4:
   jb P0.6, L5
   mov a, #'7'
   ret
L5:
   jb P0.7, L6
   mov a, #'*'
   ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad

```
L6:
   mov P0, #11111011B
   jb P0.4, L7
   mov a, #'2'
   ret
L7:
   jb P0.5, L8
   mov a, #'5'
   ret
L8:
   jb P0.6, L9
   mov a, #'8'
   ret
L9:
   jb P0.7, L10
   mov a, #'0'
   ret
```

```
L10:
   mov P0, #11111101B
   jb P0.4, L11
   mov a, #'3'
   ret
L11:
   jb P0.5, L12
   mov a, #'6'
   ret
L12:
   jb P0.6, L13
   mov a, #'9'
   ret
L13:
   jb P0.7, L14
   mov a, #'#'
   ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Scanning a Keypad

```
L14:
   mov P0, #11111110B
   jb P0.4, L15
   mov a, #'A'
   ret
L15:
   jb P0.5, L16
   mov a, #'B'
   ret
L16:
   jb P0.6, L17
   mov a, #'C'
   ret
L17:
   jb P0.7, L0
   mov a, #'D'
   ret
```

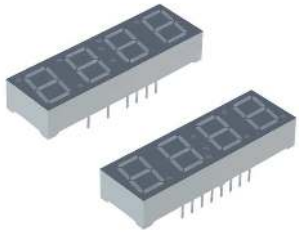

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Multiplexed Seven Segment LED displays

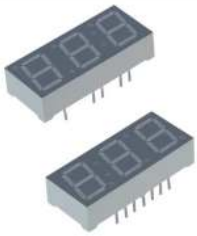
- In one of the previous lectures there is an example in which we connected two 7-segment displays to an 8051. It used two 8-bit ports.
- What if we want to add several 7-segment displays? Several options:
  - Expand the ports.
  - Add external-memory-mapped ports.
  - Add SPI or I^2C ports.
  - Multiplex <u>one</u> port. This is a technique that uses the "Persistence of vision" in people. From wikipedia: "**Persistence of vision** is the phenomenon of the eye by which an afterimage is thought to persist for approximately one twenty-fifth of a second on the retina. "

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Multiplexed Displays



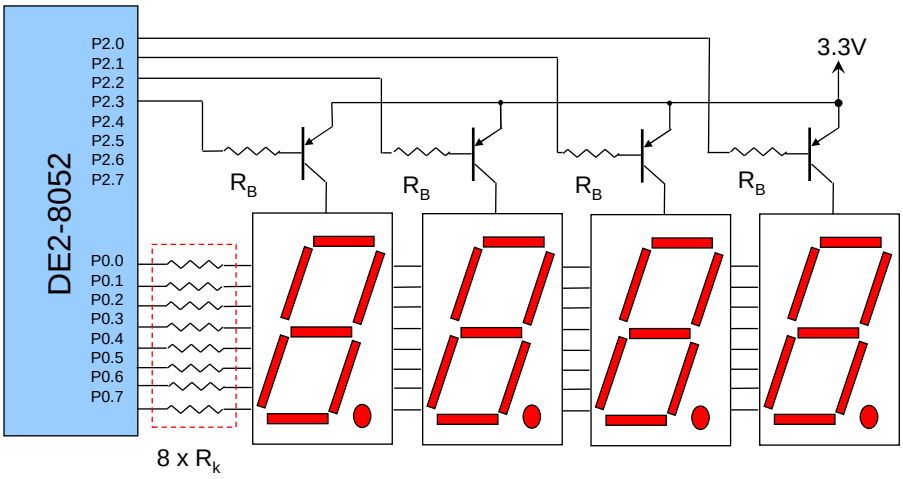
LTC-4627JR



LTC-4624JS

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Multiplexed Displays



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Multiplexed Displays: Code

- Write a program to display the 4-digit BCD number stored in registers R6-R7 to the multiplexed 7-segment display from slide 23.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Multiplexed Displays: code

```
$MODDE2
org 0
   ljmp mycode

; Look-up table (see lecture 11)...
myLUT:
    DB 0C0H, 0F9H, 0A4H, 0B0H, 099H
    DB 092H, 082H, 0F8H, 080H, 090H

Wait5ms:
;11.11MHz, 1 clk per cycle: 0.09us
L3: mov R1, #74
L2: mov R0, #250
L1: djnz R0, L1 ;3*250*0.09us=67.5us
    djnz R1, L2 ;74*67.5us=5ms
ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: code

```
mycode:
    mov SP, #7FH
    mov P0MOD, #0FFH ; All P0 pins are outputs
    mov P2MOD, #00FH ; P2.0 to P2.3 are outputs
    mov dptr, #myLUT
    mov P2, #0FH
    mov R6, #43H ; Test value
    mov R7, #21H ; Test value
forever:
; Display Digit 0
    mov A, R7
    anl A, #0FH
    movc A, @A+dptr
    mov P0, A
    clr P2.0
    lcall Wait5ms
    setb P2.0
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: code

```
; Display Digit 2
    mov A, R7
    swap A
    anl A, #0FH
    movc A, @A+dptr
    mov P0, A
    clr P2.1
    lcall Wait5ms
    setb P2.1
; Display Digit 3
    mov A, R6
    anl A, #0FH
    movc A, @A+dptr
    mov P0, A
    clr P2.2
    lcall Wait5ms
    setb P2.2
```

```
; Display Digit 4
    mov A, R6
    swap A
    anl A, #0FH
    movc A, @A+dptr
    mov P0, A
    clr P2.3
    lcall Wait5ms
    setb P2.3
; Repeat...
    sjmp forever
END
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

- The DE2-8052 soft processor has eight 7-segment displays accessible through SFRs HEX0 to HEX7 (output only ports, don't need configuration). Write an assembly clock program that displays the time of day as hours, minutes, and seconds.
- Modify the program from the previous exercise so that the hour, minute, and second displays can be set using push buttons KEY1, KEY2, and KEY3.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

- Modify the assembly program above to display a 4-digit BCD number that increments every second.
- The DE2-8052 soft processor includes a 16 x 2 (Hitachi hd44780 compatible) LCD, accessible through these SFRs (as defined in file *MODDE2*):

```
LCD_CMD   DATA 0D8H
LCD_DATA  DATA 0D9H
LCD_MOD   DATA 0DAH ; Write 0xff to make LCD_DATA an output
LCD_RW    BIT  0D8H ; '0' writes to LCD
LCD_EN    BIT  0D9H ; Toggle from '1' to '0'
LCD_RS    BIT  0DAH ; '0' for commands, '1' for data
LCD_ON    BIT  0DBH ; Write '1' to power the LCD
LCD_BLON  BIT  0DCH ; Write '1' to turn on back light
```

Write an assembly program that writes "Hello, world!" to the second line of the LCD. To help your with this task check program '*LCD_A.asm*' provided in the course web page.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.